CHECKBOX Y RADIOBUTTON EN SWING

JCheckBox, JRadioButton y el patrón Modelo-Vista-Controlador

MATERIAL DE CLASE · DESARROLLO DE SISTEMAS ORIENTADO A OBJETOS

Esteban Santini · 2026

AGENDA DE LA CLASE

01 · JCHECKBOX: SELECCIÓN MÚLTIPLE

Qué es un JCheckBox, cómo escuchar sus cambios y por qué no comparte un modelo de datos.

02 · JRADIOBUTTON: SELECCIÓN ÚNICA

El componente para elegir una única opción entre varias, agrupado con ButtonGroup.

03 · EJEMPLO 1: INGREDIENTES DE UN PEDIDO

Varios JCheckBox independientes armando el total de un pedido con extras opcionales.

04 · EJEMPLO 2: MÉTODO DE PAGO

JRadioButton agrupados calculando el total con recargo según el método elegido.

JCHECKBOX: SELECCIÓN MÚLTIPLE

javax.swing.JCheckBox — activar o desactivar opciones independientes

- Representa una opción de tipo sí/no: el usuario puede marcarla o desmarcarla en cualquier momento, sin afectar a las demás.
- A diferencia de JList o JComboBox, no tiene un modelo de datos compartido: cada JCheckBox administra su propio estado con isSelected().
- Se usan varios JCheckBox independientes cuando las opciones no son excluyentes entre sí, por ejemplo elegir varios extras de un pedido.
- setSelected(boolean) permite fijar el estado desde el código, por ejemplo para marcar una opción por defecto.
- Conviene agruparlos visualmente en un JPanel con su LayoutManager, aunque eso no afecta en nada su comportamiento lógico.

COMPONENTES RELACIONADOS

JCheckBox trabaja siempre junto con:

```
ItemListener      → detectar cambios
ButtonModel       → estado interno
JPanel            → disposición visual
import javax.swing.*;
import java.awt.event.*;
```

JCHECKBOX: ITEMLISTENER vs ACTIONLISTENER

ITEMLISTENER

- Se escucha con addItemListener(ItemListener).
- El método a implementar es itemStateChanged(ItemEvent e).
- e.getStateChange() indica ItemEvent.SELECTED o ItemEvent.DESELECTED: siempre sabés hacia dónde cambió.
- Es la opción más precisa para checkboxes: reacciona igual al marcar que al desmarcar.

ACTIONLISTENER

- Se escucha con addActionListener(ActionListener), igual que en un JButton.
- El método a implementar es actionPerformed(ActionEvent e).
- Se dispara con cada clic, pero no indica el nuevo estado: hay que consultar isSelected() aparte.
- Útil si ya usás ActionListener en el resto de la interfaz y preferís mantener un solo tipo de listener.

JCHECKBOX: CÓDIGO BÁSICO

- Se instancia con el texto que va a mostrar al lado del cuadrado.
- isSelected() devuelve el estado actual en cualquier momento, sin necesidad de un evento.
- El ItemEvent que recibe el listener también expone getItem(): el propio componente que cambió.
- setActionCommand() identifica el checkbox con un texto propio, útil cuando varios comparten un mismo listener.

```
// Instanciar
JCheckBox cbJamon = new JCheckBox("Jamón");

// Escuchar cambios
cbJamon.addItemListener(e -> {
    boolean marcado =
        e.getStateChange() == ItemEvent.SELECTED;
    System.out.println("Jamón: " + marcado);
});

// Componer
panel.add(cbJamon);
```

JRADIOBUTTON: SELECCIÓN ÚNICA

javax.swing.JRadioButton — elegir una sola opción entre varias excluyentes

- Visualmente es un círculo que se marca con un punto: la metáfora clásica de 'elegí una sola opción'.
- Por sí solo se comporta igual que un JCheckBox: nada impide marcar varios al mismo tiempo.
- La exclusividad la aporta un objeto aparte, ButtonGroup, que coordina a todos los radiobuttons que agrupa.
- ButtonGroup no es un componente visual: no se agrega a la ventana, solo administra el estado lógico del grupo.
- Se sigue necesitando un JPanel con su LayoutManager para decidir cómo se ven distribuidos en pantalla.

COMPONENTES RELACIONADOS

JRadioButton trabaja siempre con:

```
ButtonGroup      → exclusividad lógica
ActionListener   → detectar la elección
JPanel           → disposición visual
import javax.swing.*;
import java.awt.event.*;
```

JRADIOBUTTON Y BUTTONGROUP

CON BUTTONGROUP

- grupo.add(radioButton) suma cada opción al mismo grupo lógico.
- Al marcar un radiobutton, ButtonGroup desmarca automáticamente a los demás del grupo.
- En todo momento hay como máximo una opción marcada: el comportamiento que el usuario espera.
- Es la forma correcta de modelar 'el usuario elige una entre varias', como el método de pago.

SIN BUTTONGROUP

- Cada JRadioButton administra su propio estado, sin relación alguna con los demás.
- Nada impide que el usuario marque dos o más opciones que deberían ser excluyentes.
- El bug no se nota en pantalla: aparece recién cuando el código intenta leer 'la' opción elegida y encuentra varias.
- Es un error común olvidarse del ButtonGroup, sobre todo si se viene de trabajar con JCheckBox.

ButtonGroup resuelve solo la exclusividad lógica; la disposición visual (uno al lado del otro, en columna) sigue a cargo de un JPanel con su LayoutManager, exactamente igual que con JCheckBox.

JRADIOBUTTON: CÓDIGO BÁSICO

- ButtonGroup no tiene addItem(): los radiobuttons se agregan de a uno con add().
- setActionCommand() asigna una etiqueta estable a cada opción, independiente del texto visible.
- grupo.getSelection() devuelve el ButtonModel marcado, no el JRadioButton: por eso se combina con getActionCommand().
- setSelected(true) en uno de ellos define cuál queda marcado por defecto al abrir la ventana.

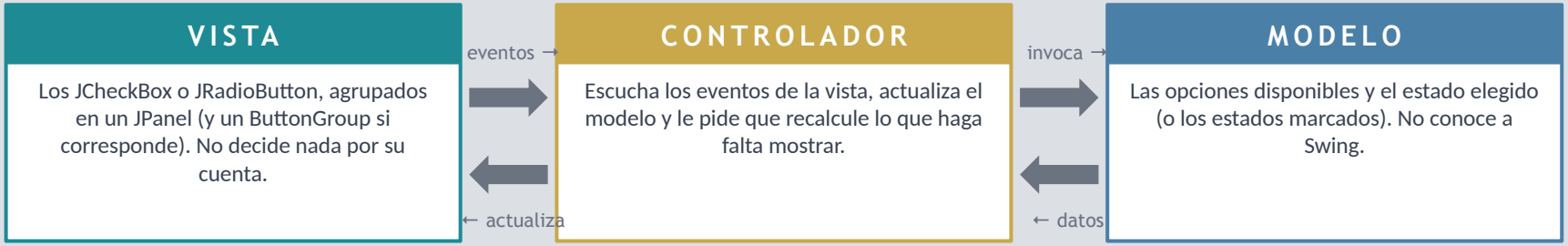
```
// Instanciar y agrupar
JRadioButton rbEfectivo = new
JRadioButton("Efectivo");
JRadioButton rbTarjeta = new JRadioButton("Tarjeta");
ButtonGroup grupo = new ButtonGroup();
grupo.add(rbEfectivo);
grupo.add(rbTarjeta);

// Identificar cada opción
rbEfectivo.setActionCommand("EFECTIVO");
rbTarjeta.setActionCommand("TARJETA");

// Leer la selección vigente
String elegido = grupo.getSelection()
    .getActionCommand();
```

ARQUITECTURA MVC APLICADA

Aplicamos el mismo esquema Modelo–Vista–Controlador de siempre, ahora con JCheckBox y JRadioButton como parte de la vista.



- Regla de oro (la misma de siempre): el modelo nunca importa javax.swing.
- El controlador es el único que conecta los componentes de selección con el modelo: ninguno de los dos se conoce directamente.
- Vamos a construir dos ejemplos completos: uno con JCheckBox (ingredientes de un pedido) y otro con JRadioButton (método de pago).

EJEMPLO 1: INGREDIENTES DE UN PEDIDO

Varios JCheckBox independientes, cada uno con su propio precio

- Cada ingrediente disponible se muestra como un JCheckBox independiente: Muzzarella extra, Jamón, Champiñones, Aceitunas y Panceta.
- El usuario puede marcar cualquier combinación, incluidas todas o ninguna: no hay exclusividad entre ellos.
- Cada vez que se marca o desmarca un ingrediente, se recalcula el total de extras y se muestra en pantalla.
- Lo vamos a construir con cuatro clases: Ingrediente, PedidoModelo, PedidoVista y PedidoControlador.

INGREDIENTES DISPONIBLES

Cada uno con su precio:

```
Muzzarella extra →  $400
Jamón            →  $500
Panceta          →  $600
List<Ingrediente>
selección = Set<String>
```

MVC EN SWING (PEDIDO) — LA CLASE INGREDIENTE

- Ingrediente es un objeto de dominio simple: nombre y precio, sin ningún import de Swing.
- Es inmutable: sus dos campos son final y se fijan una sola vez en el constructor.
- getPrecio() devuelve un double, que después se usa para sumar el total de extras elegidos.
- Esta misma clase podría reutilizarse en cualquier otra parte del sistema que necesite representar un ingrediente.

```
// Ingrediente: una opción de extra (nombre y precio)
public class Ingrediente {

    private final String nombre;
    private final double precio;

    public Ingrediente(String nombre,
                       double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }
}
```

MVC EN SWING (PEDIDO) — EL MODELO

- disponibles es la lista fija de ingredientes que existen para elegir: no cambia en tiempo de ejecución.
- seleccionados guarda solo los nombres marcados en este momento, en un Set<String> sin duplicados.
- alternar() es el único punto de entrada para modificar la selección: agrega o quita según el nuevo estado del checkbox.
- calcularTotalExtras() no sabe nada de JCheckBox: solo filtra y suma precios, con Streams.

```
import java.util.*;

// MODELO: ingredientes disponibles y selección actual
public class PedidoModelo {

    private final List<Ingrediente> disponibles = List.of(
        new Ingrediente("Muzzarella extra", 400),
        new Ingrediente("Jamón", 500),
        new Ingrediente("Champiñones", 350),
        new Ingrediente("Aceitunas", 300),
        new Ingrediente("Panceta", 600));
    private final Set<String> seleccionados = new HashSet<>();

    public List<Ingrediente> obtenerDisponibles() {
        return disponibles;
    }

    public void alternar(String nombre, boolean marcado) {
        if (marcado) seleccionados.add(nombre);
        else seleccionados.remove(nombre);
    }

    public double calcularTotalExtras() {
        return disponibles.stream()
            .filter(i -> seleccionados.contains(i.getNombre()))
            .mapToDouble(Ingrediente::getPrecio)
            .sum();
    }
}
```

MVC EN SWING (PEDIDO) — LA VISTA

- Recibe la lista de ingredientes disponibles por parámetro: no decide ella misma qué opciones existen.
- Crea un JCheckBox por ingrediente, con su nombre y precio en el texto, y guarda todos en una lista propia.
- setActionCommand() identifica cada checkbox con el nombre exacto del ingrediente, sin depender del texto visible.
- Expone getChecks() y mostrarTotal(): el controlador los usa para escuchar y para actualizar la pantalla.

```
import javax.swing.*;
import java.awt.*;
import java.util.List;
import java.util.ArrayList;
// VISTA: un JCheckBox por ingrediente, sin lógica
public class PedidoVista extends JFrame {

    private final JPanel panel = new JPanel(new GridLayout(0, 1));
    private final JLabel labelTotal = new JLabel("Extras: $0");
    private final List<JCheckBox> checks = new ArrayList<>();

    public PedidoVista(List<Ingrediente> disponibles) {
        super("Ingredientes del pedido");
        for (Ingrediente ing : disponibles) {
            JCheckBox cb = new JCheckBox(ing.getNombre()
                + " (+$" + (int) ing.getPrecio() + ")");
            cb.setActionCommand(ing.getNombre());
            checks.add(cb); panel.add(cb);
        }
        setLayout(new BorderLayout(10, 10));
        add(panel, BorderLayout.CENTER);
        add(labelTotal, BorderLayout.SOUTH);
        setSize(340, 260);
        setLocationRelativeTo(null);
setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
    public List<JCheckBox> getChecks() { return checks; }
    public void mostrarTotal(double total) {
        labelTotal.setText("Extras: $" + (int) total);
    }
}
```

MVC EN SWING (PEDIDO) — EL CONTROLADOR

```
import javax.swing.*;

// CONTROLADOR: conecta la vista con el modelo
public class PedidoControlador {

    private final PedidoModelo modelo;
    private final PedidoVista vista;

    public PedidoControlador(PedidoModelo modelo, PedidoVista vista) {
        this.modelo = modelo; this.vista = vista;
        for (JCheckBox cb : vista.getChecks()) {
            cb.addItemListener(e -> actualizarTotal(cb, cb.isSelected()));
        }
    }

    private void actualizarTotal(JCheckBox cb, boolean marcado) {
        modelo.alternar(cb.getActionCommand(), marcado);
        vista.mostrarTotal(modelo.calcularTotalExtras());
    }
}
```

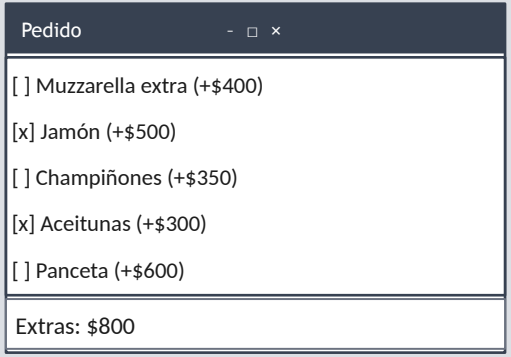
- Registra un ItemListener por cada checkbox de la vista: cualquiera de los cinco dispara el mismo método.
- actualizarTotal() lee el ActionCommand del checkbox que cambió, no su texto completo (que incluye el precio).
- El modelo decide si suma o resta ese ingrediente; el controlador solo le pide que recalcule el total.
- El main arma las tres piezas y muestra la ventana, dentro de SwingUtilities.invokeLater(), como siempre.

RESULTADO EN EJECUCIÓN — PEDIDO

```
// PUESTA EN MARCHA
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        PedidoModelo modelo = new PedidoModelo();
        PedidoVista vista = new PedidoVista(
            modelo.obtenerDisponibles());
        new PedidoControlador(modelo, vista);
        vista.setVisible(true);
    });
}
```

El main solo instancia las tres clases y muestra la ventana.
Cada clic en un checkbox dispara actualizarTotal(), que recalcula el total consultando al modelo.

RESULTADO



EJEMPLO 2: MÉTODO DE PAGO

JRadioButton agrupados con ButtonGroup, selección única

- Cuatro métodos de pago posibles, mutuamente excluyentes: Efectivo, Débito, Crédito y Transferencia.
- Cada método tiene su propio recargo porcentual: por ejemplo, pagar con crédito suma un 10% sobre el monto base.
- Al elegir un método, se recalcula el total a pagar sobre un monto base fijo y se muestra en pantalla.
- Lo vamos a construir con cuatro clases: MetodoPago, PagoModelo, PagoVista y PagoControlador.

MÉTODOS DISPONIBLES

Cada uno con su recargo:

```
Efectivo        →  0% recargo
Crédito         →  10% recargo
Transferencia   →  0% recargo
List<MetodoPago>
selección = 1 sola opción
```

MVC EN SWING (PAGO) — LA CLASE METODOPAGO

- MetodoPago es un objeto de dominio simple: nombre y recargo porcentual, sin ningún import de Swing.
- Es inmutable, igual que Ingrediente: sus dos campos son final y se fijan una sola vez en el constructor.
- getRecargoPorcentaje() devuelve un double que después se usa para calcular el total a pagar.
- El mismo patrón de clase de dominio simple se repite en los dos ejemplos: Ingrediente y MetodoPago.

```
// MetodoPago: una opción de pago (nombre y recargo)
public class MetodoPago {
    private final String nombre;
    private final double recargoPorcentaje;

    public MetodoPago(String nombre,
                      double recargoPorcentaje) {
        this.nombre = nombre;
        this.recargoPorcentaje = recargoPorcentaje;
    }

    public String getNombre() { return nombre; }
    public double getRecargoPorcentaje() {
        return recargoPorcentaje;
    }
}
```

MVC EN SWING (PAGO) — EL MODELO

- metodos es la lista fija de opciones disponibles; montoBase es el importe sobre el que se calcula el recargo.
- seleccionado guarda el MetodoPago elegido en este momento; arranca en la primera opción de la lista.
- seleccionar() busca el método por nombre y lo asigna, igual que alternar() hacía con los ingredientes.
- calcularTotal() no sabe nada de JRadioButton: aplica el porcentaje de recargo sobre el monto base.

```
import java.util.List;

// MODELO: métodos disponibles, monto base y elegido
public class PagoModelo {

    private final List<MetodoPago> metodos = List.of(
        new MetodoPago("Efectivo", 0),
        new MetodoPago("Débito", 0),
        new MetodoPago("Crédito", 10),
        new MetodoPago("Transferencia", 0));
    private final double montoBase = 10000;
    private MetodoPago seleccionado = metodos.get(0);

    public List<MetodoPago> obtenerMetodos() {
        return metodos;
    }

    public void seleccionar(String nombre) {
        metodos.stream()
            .filter(m -> m.getNombre().equals(nombre))
            .findFirst()
            .ifPresent(m -> seleccionado = m);
    }

    public double calcularTotal() {
        double recargo = seleccionado.getRecargoPorcentaje();
        return montoBase * (1 + recargo / 100);
    }
}
```

MVC EN SWING (PAGO) — LA VISTA

- Recibe la lista de métodos por parámetro y crea un JRadioButton por cada uno, todos dentro del mismo ButtonGroup.
- botones guarda cada JRadioButton en una lista propia, igual que checks lo hacía con los JCheckBox del pedido.
- setActionCommand() identifica cada radiobutton con el nombre exacto del método, sin depender del texto visible.
- Expone getGrupo() y getBotones(): el controlador los necesita para leer y para escuchar la selección vigente.

```
import javax.swing.*;
import java.awt.*;
import java.util.List;
import java.util.ArrayList;
// VISTA: un JRadioButton por método, con ButtonGroup
public class PagoVista extends JFrame {

    private final ButtonGroup grupo = new ButtonGroup();
    private final JPanel panel = new JPanel(new GridLayout(0, 1));
    private final List<JRadioButton> botones = new ArrayList<>();
    private final JLabel labelTotal = new Jlabel();

    public PagoVista(List<MetodoPago> metodos) {
        super("Método de pago");
        for (MetodoPago m : metodos) {
            JRadioButton rb = new JRadioButton(m.getNombre()
                + " (+" + (int) m.getRecargoPorcentaje() + "%)");
            rb.setActionCommand(m.getNombre());
            grupo.add(rb); botones.add(rb); panel.add(rb);
        }
        botones.get(0).setSelected(true);
        setLayout(new BorderLayout(10, 10));
        add(panel, BorderLayout.CENTER);
        add(labelTotal, BorderLayout.SOUTH);
        setSize(340, 260);
        setLocationRelativeTo(null); setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
    public ButtonGroup getGrupo() { return grupo; }
    public List<JRadioButton> getBotones() { return botones; }
    public void mostrarTotal(double total) {
        labelTotal.setText("Total a pagar: $" + (int) total);
    }
}
```

MVC EN SWING (PAGO) — EL CONTROLADOR

```
import javax.swing.*;

// CONTROLADOR: conecta la vista con el modelo
public class PagoControlador {

    private final PagoModelo modelo;
    private final PagoVista vista;

    public PagoControlador(PagoModelo modelo, PagoVista vista) {
        this.modelo = modelo; this.vista = vista;
        for (JRadioButton rb : vista.getBotones()) {
            rb.addActionListener(e -> actualizarTotal());
        }
        actualizarTotal();
    }
    private void actualizarTotal() {
        String elegido = vista.getGrupo().getSelection().getActionCommand();
        modelo.seleccionar(elegido);
        vista.mostrarTotal(modelo.calcularTotal());
    }
}
```

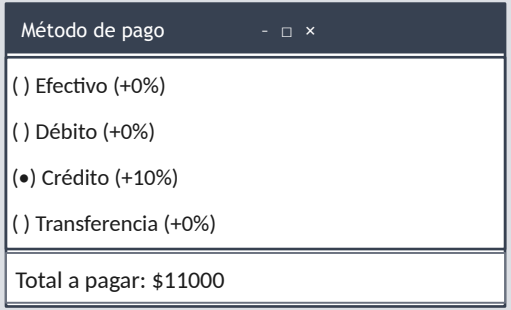
- Registra un ActionListener por cada radiobutton de la vista: cualquiera de los cuatro dispara el mismo método.
- actualizarTotal() no le pregunta nada al radiobutton que cambió: le pregunta al grupo cuál quedó marcado.
- grupo.getSelection() devuelve el ButtonModel activo; getActionCommand() traduce eso al nombre del método.
- El main arma las tres piezas y muestra la ventana, dentro de SwingUtilities.invokeLater(), como siempre.

RESULTADO EN EJECUCIÓN — PAGO

```
// PUESTA EN MARCHA
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        PagoModelo modelo = new PagoModelo();
        PagoVista vista = new PagoVista(
            modelo.obtenerMetodos());
        new PagoControlador(modelo, vista);
        vista.setVisible(true);
    });
}
```

El main solo instancia las tres clases y muestra la ventana.
Cada clic en un radiobutton dispara actualizarTotal(), que consulta al grupo cuál quedó marcado.

RESULTADO



BUENAS PRÁCTICAS Y ERRORES COMUNES

BUENAS PRÁCTICAS

- Usar ButtonGroup para cualquier conjunto de opciones mutuamente excluyentes, nunca dejarlo librado al usuario.
- Identificar cada checkbox o radiobutton con setActionCommand(), en vez de parsear su texto visible.
- Actualizar el modelo primero y pedirle después el nuevo total o estado: la vista nunca calcula nada.
- Construir toda la interfaz dentro de SwingUtilities.invokeLater(), como cualquier aplicación Swing.

ERRORES COMUNES

- Agrupar visualmente radiobuttons en un JPanel y olvidarse del ButtonGroup: siguen siendo independientes.
- Leer cb.getText() para identificar un checkbox, en vez de un ActionCommand estable.
- Confundir ItemListener (marca/desmarca) con ActionListener (clic) y perder de vista alguno de los dos estados.
- Guardar el estado elegido en la vista, por ejemplo un campo booleano ahí, en lugar de delegarlo al modelo.

Son los mismos principios que ya vimos con JList, JComboBox y JTable: el componente visual nunca guarda la verdad de los datos, solo la refleja.

RESUMEN Y BUENAS PRÁCTICAS

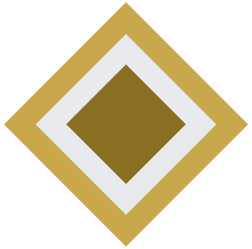
- JCheckBox modela opciones independientes; JRadioButton modela una elección única, agrupado con ButtonGroup.
- Ninguno de los dos tiene un modelo de datos compartido como ListModel o TableModel: cada uno administra su propio isSelected().
- setActionCommand() es la forma prolija de identificar cada opción, en vez de depender del texto visible.
- La arquitectura MVC se aplica igual que siempre: modelo sin Swing, vista sin lógica de negocio, controlador coordinando.
- Los dos ejemplos completos (Pedido y Pago) quedan como referencia para la práctica.

SIDE QUEST

Práctica: agregá una opción condicionada.

Un JCheckBox 'Envío exprés' que solo se pueda marcar una vez elegido un método de pago: deshabilitalo con setEnabled(false) hasta ese momento.

Documentación: docs.oracle.com → How to Use Buttons, Check Boxes, and Radio Buttons



MUCHAS GRACIAS

"Marcar una casilla es una decisión; elegir una opción entre varias es otra: el componente que uses debería dejarlo claro de un vistazo."

Preguntas y consultas: ahora, o por los canales de la materia.
En la próxima clase: práctica integradora de Swing + MVC.